Carbon-Aware Container Orchestration: A Comparative Analysis of Heuristic and Optimal Scheduling Algorithms for Sustainable Computing

Nasir Asadov, Vlad C. Coroamă, Marco Zambianco, Silvio Cretti, and Matthias Finkbeiner

***Abstract*—Container orchestration systems ignore carbon emissions in placement decisions, despite data centers consuming over 1% of global electricity. Existing Kubernetes schedulers optimize for performance but lack carbon minimization mechanisms. We formulate the carbon-aware scheduling problem and develop a heuristic algorithm that incorporates both operational and embodied carbon emissions while respecting resource requirements and deadlines. Evaluation against vanilla Kubernetes and theoretical optimal solutions demonstrates our algorithm reduces carbon emissions significantly while maintaining practical execution times. This establishes that carbon-aware container orchestration provides a feasible pathway toward sustainable cloud computing.**

***Index Terms*—Carbon-aware computing, container orchestration, Kubernetes, sustainable computing, green cloud computing, optimization algorithms, mixed integer linear programming, global optimization.**

I. Introduction

A. Motivation

THE exponential growth of cloud computing has led to significant environmental concerns, with data centers consuming approximately 1% of global electricity and contributing substantially to carbon emissions [1]. As organizations increasingly adopt containerized applications and orchestration platforms like Kubernetes, the need for carbon-aware scheduling mechanisms becomes paramount to address sustainability challenges in modern computing infrastructures.

B. Contributions

- We formulate the carbon-aware scheduling problem and develop a heuristic algorithm that incorporates both operational and embodied carbon emissions while respecting resource requirements and deadlines.
- We evaluate our algorithm against vanilla Kubernetes and theoretical optimal solutions demonstrating significant carbon emission reductions while maintaining practical execution times.
- We establish that carbon-aware container orchestration provides a feasible pathway toward sustainable cloud computing.

N. Asadov, V. C. Coroamă, and M. Finkbeiner are with the Department of Environmental Technology, Technische Universität Berlin, Strasse des 17. Juni 135, 10623 Berlin, Germany (e-mail: nasir.asadov@tu-berlin.de; coroama@tu-berlin.de; matthias.finkbeiner@tu-berlin.de).

M. Zambianco and S. Cretti are with Fondazione Bruno Kessler, 38123 Trento, Italy (e-mail: zambianco@fbk.eu; cretti@fbk.eu).

Corresponding author: Nasir Asadov (e-mail: nasir.asadov@tu-berlin.de).

Manuscript received Month Day, Year; revised Month Day, Year.

C. Research Questions

RQ1: How does the proposed algorithm compare to baseline algorithms in terms of carbon emission reduction?

RQ2: How does the proposed algorithm compare to baseline algorithms in terms of computational complexity?

RQ3: How does the proposed algorithm compare to baseline algorithms in terms of placement success rate?

II. Related Work

A. Summary of Paper I

Our earlier review analyzed carbon-aware workload placement and scheduling along nine axes: (i) temporal shifting; (ii) spatial shifting; (iii) network and migration energy; (iv) which carbon signal is used (average vs. marginal); (v) forecasting of demand and carbon signals; (vi) forecasting targets and approaches; (vii) workload-to-power mapping; (viii) inclusion of embodied/life-cycle emissions; and (ix) code availability for reproducibility. We found that most systems optimized either time *or* place, rarely both; average carbon intensity (ACI) dominated over marginal signals; network/migration energy was often ignored; forecasting was seldomly applied; embodied emissions were virtually absent; and open-sourced codebases was sporadic. These gaps motivated the benchmarking choices in this paper.

B. Developments Since Paper I (2025)

Temporal Shifting with Explicit Power Mapping: GREEN [2] schedules ML training jobs to align cluster power draw with lower carbon-intensity (CI) windows while preserving daily capacity; it couples temporal shifting with per-job power modeling (GPU via NVIDIA Management Library, NVML; CPU model), reporting substantial cluster-wide CO_2e reductions at small job completion time (JCT) trade-offs. This directly addresses our temporal-shifting and power-mapping axes and provides scheduler-overhead/latency numbers useful for benchmarks.

Spatial (and Spatiotemporal) Shifting under SLOs: At the edge, CarbonEdge [3] demonstrates that carbon intensity varies at mesoscale granularity (within states/among neighboring countries), enabling geography-aware placement that respects tight latency budgets—i.e., spatial shifting with explicit service-level objective (SLO) constraints. For always-on services that cannot move across regions, Quality Time

[4] uses *forecast-based* quality adaptation within a region to reduce CO_2e while meeting service constraints, giving an orthogonal baseline along our forecasting axis.

Signals and Benchmarking: `ElectricityEmissions.jl` [5] provides a reproducible toolkit to compute and compare multiple carbon signals (ACI, marginal variants, and a proposed hybrid) on standard grid cases, showing how signal choice can flip measured outcomes. Complementing this, a 2025 statement paper [6] argues that *marginal carbon intensity (MCI)* is neither reliable nor actionable for accounting or grid flexibility due to non-observability and unverifiability; it advocates using more actionable signals such as directly reported "excess renewable power," and incorporating storage and stability explicitly in marginal metrics.

Network/Migration Energy: Beyond compute placement, LINTS [7] schedules inter-datacenter *data transfers* over time to exploit carbon-intensity (CI) variability along the transfer path, explicitly modeling network energy and meeting deadlines—filling a gap our review identified around transfer-dominated workloads.

Embodied Emissions: Finally, AGING-AWARE CPU CORE MANAGEMENT [8] introduces a run-time policy that *amortizes embodied emissions* by extending CPU lifetime in LLM inference clusters, quantifying when embodied-aware decisions change the operational optimum. This is directly relevant to our life-cycle axis and to reporting operational vs. embodied sensitivity in benchmarks.

C. Embodied Emissions Allocation in Carbon-aware Scheduling

Embodied emissions (manufacturing, logistics, and end-of-life) are increasingly recognized as a first-order term in carbon-aware scheduling, yet are often omitted or uniformly amortized over calendar time. Two broad allocation strategies appear in the literature: (i) lifetime amortization (time-based), and (ii) utilization-aware attribution that splits embodied carbon by actual time-in-use and resource share.

Standards and frameworks: The Green Software Foundation's Software Carbon Intensity (SCI) specification formalizes embodied allocation with

$$M = TE \times TS \times RS,$$

where TE is total device embodied emissions from LCA, TS is the time share (e.g., reserved or in-use fraction of expected lifetime), and RS is the resource share (e.g., vCPU, memory) attributed to the software component [9], [10]. This structure subsumes both simple time-based amortization ($RS = 1$) and finer-grained attribution when detailed utilization or reservations are available.

Recent research: Beyond uniform amortization, several works incorporate embodied carbon into scheduling and hardware-management decisions: (i) Ji *et al.* propose carbon depreciation models that allocate a larger share of embodied carbon to newly provisioned servers and recover idle-time impacts during active use, showing that traditional accounting can inadvertently favor premature refresh [11]; (ii) Hewage *et al.* demonstrate aging-aware CPU core management that extends lifetime and reduces embodied-attributed emissions in inference clusters with minimal QoS impact [8]; (iii) Zhang, Song, and Sahoo formulate joint embodied+operational objectives with heterogeneity and age in dispatch, yielding lower total emissions under utilization-aware lifetime models [12]; and (iv) for edge contexts, Panteleaki *et al.* advocate co-optimizing embodied and operational carbon under intermittent utilization and tight resource constraints [13].

Implications for scheduling: (1) Pure calendar-based amortization is appropriate when only coarse information is available, but it obscures idle periods and can inflate per-workload embodied intensity. (2) With utilization traces (e.g., hourly CPU), allocating embodied emissions in proportion to time-in-use and resource share yields more faithful attribution and can shift policies toward reusing capacity, consolidating strategically, or delaying refresh. (3) Accounting for device age and heterogeneity reveals that newer hardware, despite higher performance-per-watt, carries substantial embodied carbon that must be amortized through sustained use before outperforming older devices on total emissions. (4) These effects are amplified at the edge, where intermittency and under-utilization are common.

In this paper, we adopt a refined, SCI-consistent allocation: we attribute embodied emissions using time-in-use and resource share derived from hourly CPU utilization, and we report results alongside a lifetime-amortization baseline to quantify how allocation choice affects scheduler decisions and total carbon efficiency.

D. Takeaways

The 2025 literature advances the axes flagged in our review: temporally aware schedulers with measured power models (GREEN); spatial shifting at finer granularity with SLOs (CARBONEDGE); forecast-driven controllers for always-on services (QUALITY TIME); reproducible signal computation and guidance on actionable metrics beyond MCI (`ElectricityEmissions.jl`, [6]); explicit accounting of network transfer CO_2e (LINTS); and first steps toward embodied-aware *operational* policies (aging-aware CPU management). These works close several gaps we highlighted and provide solid guidance for our benchmarking protocol.

III. PROPOSED METHOD

A. The Hidden Impact: Why Embodied Emissions Matter

Current carbon-aware scheduling approaches in the literature focus predominantly on operational emissions—the carbon footprint from electricity consumed during workload execution. While this operational perspective captures the immediate energy consumption of running containers, it fundamentally overlooks a critical component of the total carbon footprint: embodied emissions from hardware manufacturing, transportation, and end-of-life processing.

Our preliminary analysis reveals that embodied emissions constitute a substantial portion of the total carbon footprint in modern data center environments. Based on detailed lifecycle assessment of representative hardware configurations from

[14], we find that embodied carbon represents a significant fraction of total emissions over typical hardware lifetimes, with this proportion varying significantly based on hardware type, utilization patterns, and regional electricity carbon intensity.

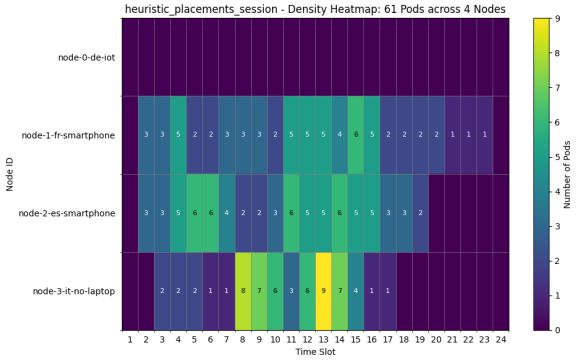

Fig. 1. Scheduling decisions using operational emissions only. The algorithm optimizes placement based solely on electricity carbon intensity, leading to concentrated utilization patterns that may overlook hardware embodied carbon implications.

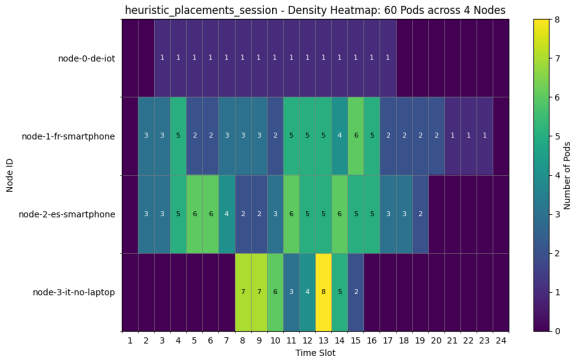

Fig. 2. Scheduling decisions incorporating both operational and embodied emissions. The comprehensive approach reveals markedly different placement patterns, demonstrating how embodied carbon considerations fundamentally alter the optimization landscape.

The contrast between Figures 1 and 2 illustrates a profound shift in the scheduling decision landscape when embodied emissions are incorporated. The operational-only approach (Figure 1) exhibits clustering behavior that prioritizes nodes with favorable electricity carbon intensity, often concentrating workloads on a subset of available hardware. However, when embodied emissions are considered (Figure 2), the algorithm discovers a more distributed utilization pattern that balances both operational and embodied carbon costs.

This transformation in scheduling behavior emerges from a fundamental insight: embodied emissions represent a *fixed carbon cost* that must be amortized over the hardware's operational lifetime. Unlike operational emissions that scale directly with utilization, embodied emissions create an incentive to maximize hardware utilization efficiency across the entire infrastructure. Consequently, the comprehensive approach often favors distributing workloads more evenly to amortize embodied carbon costs effectively, even when this means accepting slightly higher operational emissions in some time slots.

The implications extend beyond mere placement optimization. When embodied emissions are ignored, scheduling algorithms may inadvertently create scenarios where low-carbon electricity incentivizes concentrating workloads on a subset of nodes, leaving other hardware underutilized. This apparent optimization actually increases the per-workload embodied carbon footprint by failing to amortize manufacturing emissions across the full infrastructure capacity.

Our analysis demonstrates that neglecting embodied emissions can lead to suboptimal scheduling decisions that significantly underestimate total carbon impact in typical cloud workload scenarios. More critically, operational-only approaches may actively work against sustainability goals by creating utilization patterns that waste the embodied carbon investment in underutilized hardware.

B. System Architecture Overview

Our carbon-aware orchestration system is implemented as a distributed architecture that integrates with Kubernetes through a custom gRPC-based interface. The system decouples carbon-aware scheduling logic from the underlying orchestration platform, enabling algorithmic experimentation while maintaining production-grade performance and reliability.

Figure 3 illustrates the complete system architecture and component interactions. The system follows a layered design where external clients communicate through a standardized gRPC interface, while the core engine coordinates between algorithms, data services, and persistent state management.

1) Core System Components: The architecture comprises five interconnected components:

1. gRPC Placement Service: The central `PlacementAlgorithm` service implements the Interface Definition Language (IDL) protocol, providing a language-agnostic API for scheduling requests. The service receives infrastructure snapshots and workload specifications, returning optimized placement decisions through standardized protocol buffer messages.

2. Multi-Algorithm Engine: A pluggable algorithm framework supporting two scheduling approaches: (i) heuristic spatiotemporal optimization for real-time online scheduling of incoming pods, and (ii) MILP-based oracle optimization that assumes perfect knowledge of all pods (including future arrivals) to compute theoretically optimal solutions for benchmarking and analysis.

3. Carbon Data Service: Aggregates and processes carbon intensity forecasts from multiple data sources including Electricity Maps API, regional grid operators, and weather-based renewable energy predictions. The service maintains rolling 24-48 hour forecasts with hourly updates, supporting geographical and temporal carbon optimization across distributed infrastructure.

4. Hardware Metadata Repository: Maintains comprehensive hardware profiles including power consumption characteristics (idle, 10%, 50%, maximum load), embodied carbon values from lifecycle assessments, and expected operational

lifetimes. Node metadata is extracted from Kubernetes annotations and labels, enabling fine-grained carbon accounting per hardware category.

5. Performance Monitoring and Analytics: Implements comprehensive logging through `PerformanceLogger` and `ExperimentLogger` classes, capturing algorithm execution metrics, carbon emissions outcomes, and resource utilization patterns. The monitoring system enables continuous algorithm improvement and validation of carbon reduction claims.

2) Integration Architecture: **Kubernetes Integration:** The system interfaces with Kubernetes through the IDL protocol, which models cluster snapshots including node flavors, resource availability, and workload requirements. The gRPC interface enables polyglot implementation with Python-based algorithms and Go-based clients, facilitating integration with existing Kubernetes scheduler frameworks.

Data Flow Pipeline: Infrastructure data flows from Kubernetes node specifications through the IDL interface to algorithm implementations. Carbon intensity forecasts are loaded from JSON configuration files or real-time API endpoints. The system processes scheduling requests by combining infrastructure constraints, workload requirements, and carbon optimization objectives to produce placement decisions.

State Management: `PersistentStateStorage` maintains scheduling state across service restarts, enabling consistent placement decisions and experiment continuity. The state management layer supports both stateless operation for production deployments and stateful experiment tracking for research applications.

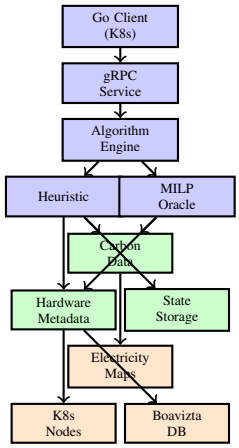

Fig. 3. Carbon-aware orchestration system architecture with component interactions and data flow.

C. Carbon Accounting Models

1) Carbon Signals: Our carbon intensity modeling incorporates both geographical and temporal variations in electricity grid carbon footprint, providing the foundation for accurate operational carbon emission calculations.

Grid-Level Forecasting: The system integrates carbon intensity forecasts from Electricity Maps API, which provides real-time and predicted carbon intensity data for European regions. Forecasts incorporate renewable energy generation, fossil fuel plant dispatch, and grid interconnection effects.

Regional Coverage: The current implementation supports four European regions (DE, FR, ES, IT-NO) with hourly carbon-intensity forecasts. Regional data shows carbon intensities ranging from 30 gCO_2/kWh to 479 gCO_2/kWh, reflecting differences in electricity generation portfolios across European grids.

Forecast Processing: The `build_timeslots` function constructs temporal scheduling windows by combining carbon intensity forecasts with infrastructure availability. Timeslots align with forecast granularity (hourly) while supporting sub-hourly workload placement through interpolation methods.

2) Power Modeling: **Multi-Point Power Models:** Each node maintains power consumption profiles at four utilization levels: idle, 10%, 50%, and maximum load. These profiles enable accurate power estimation based on actual workload characteristics rather than simplified linear models.

Dynamic Power Calculation: Power consumption for individual workloads is calculated using the formula:

$$P_{workload} = P_{idle} + (P_{max} - P_{active}) \times \frac{CPU_{request}}{CPU_{total}}$$

where P_{active} represents the 50% load power consumption, providing a realistic baseline for container workload power estimation.

Metadata Integration: Hardware power profiles are extracted from Kubernetes node annotations (`hardware.power/idle_watts`, `hardware.power/active_watts`) with fallback to category-based defaults when node-specific data is unavailable.

3) Embodied Emissions and Amortization: To accurately account for embodied emissions in our scheduling decisions, we developed a comprehensive methodology for quantifying the embodied carbon footprint of ICT hardware. Our approach addresses a critical gap in existing carbon-aware systems, which typically ignore the substantial manufacturing-related emissions embedded in computing infrastructure.

Our embodied carbon calculations are based on lifecycle assessment (LCA) data from the Boavizta database [14], which aggregates manufacturer-reported environmental product declarations following ISO 14040/14044 standards. We processed a comprehensive dataset of hardware embodied carbon values and computed category-wise averages for the device types used in our experimental evaluation, as summarized in Table I.

TABLE I
HARDWARE CATEGORY EMBODIED CARBON VALUES

Hardware Category	Embodied Emissions (kg CO_2e)	Lifetime (years)
IoT	27.47	5.20
Smartphone	52.73	3.03
Laptop	231.86	4.13
Server	1230.66	3.87

The dataset encompasses over 200 hardware configurations from major manufacturers including Apple, Dell, and others, providing representative embodied carbon values across different hardware categories. For each category, we computed

weighted averages of embodied emissions and operational lifetimes from the underlying device-specific LCA data.

The core challenge in embodied carbon accounting is properly amortizing the upfront manufacturing emissions over the hardware's operational lifetime and allocating them to individual workloads. Our model follows the temporal amortization approach:

$$E_{emb,hourly} = \frac{E_{emb,total}}{L_{lifetime} \times 365 \times 24} \tag{1}$$

where $E_{emb,total}$ represents the total embodied emissions for a hardware unit (in grams CO_2e), $L_{lifetime}$ is the expected operational lifetime in years, and $E_{emb,hourly}$ is the amortized hourly embodied carbon cost.

For workload allocation, embodied emissions are attributed proportionally to the workload duration:

$$E_{emb,workload} = E_{emb,hourly} \times t_{duration} \tag{2}$$

where $t_{duration}$ represents the workload execution time in hours. This allocation model ensures that longer-running workloads bear a proportionally larger share of the embodied carbon costs, creating appropriate incentives for efficient resource utilization.

Our implementation extracts embodied carbon metadata from Kubernetes node annotations using a hierarchical priority system. Hardware-specific values take precedence over category defaults, enabling fine-grained modeling while maintaining compatibility with heterogeneous infrastructure deployments.

The embodied carbon calculation is integrated directly into the scheduling objective function, where it combines with operational emissions to provide a comprehensive carbon footprint assessment for each placement decision. This integrated approach ensures that both emission components influence scheduling decisions simultaneously, rather than treating them as separate optimization phases.

D. Problem Formulation (MILP)

We formulate the carbon-aware scheduling problem as a Mixed Integer Linear Programming (MILP) optimization that finds optimal assignments of pods (computational tasks) to nodes (computing resources) and timeslots while minimizing total carbon emissions incorporating both operational and embodied components.

1) Sets, Parameters and Decision Variables: **Sets:** Let $i \in \mathcal{P}$ denote the set of pods, $j \in \mathcal{F}$ the set of nodes, and $t \in \mathcal{T}$ the set of timeslots.

Parameters:

$D_i \in \mathbb{N}^+$	Deadline timeslot for pod i	(3)
$s_i \in \mathbb{N}^+$	Earliest timeslot for pod i	(4)
$d_i \in \mathbb{N}^+$	Duration of pod i (in timeslots)	(5)
$w_i \in \mathbb{N}^+$	Temporal slack for pod i	(6)
$c_i \in \mathbb{R}^+$	CPU request of pod i	(7)
$r_i \in \mathbb{R}^+$	RAM request of pod i	(8)
$C_{jt} \in \mathbb{R}^+$	Available CPU of node j in timeslot t	(9)
$R_{jt} \in \mathbb{R}^+$	Available RAM of node j in timeslot t	(10)
$p_{ij} \in \mathbb{R}^+$	Power consumption of pod i on node j	(11)
$\text{CI}_{j,t} \in \mathbb{R}^+$	Carbon intensity of node j in timeslot t	(12)
$\text{EC}_j \in \mathbb{R}^+$	Embodied carbon of node j	(13)
$L_j \in \mathbb{R}^+$	Expected lifetime of node j	(14)
$u_j \in \mathbb{R}^+$	Utilization factor of node j	(15)

Decision Variables: The binary decision variable is defined as:

$$x_{ijt} = \begin{cases} 1 & \text{if pod } i \text{ placed on node } j \text{ at time } t \\ 0 & \text{otherwise} \end{cases} \tag{16}$$

Feasible Placements: Let $\mathcal{V}_i \subset \mathcal{F} \times \mathcal{T}$ be the set of valid node-timeslot pairs for pod i:

$$\begin{aligned} \mathcal{V}_i = \{(j,t) \in \mathcal{F} \times \mathcal{T} \,|t \geq s_i,\ t + d_i \leq D_i, \\ C_{j(t+k)} \geq c_i \ \forall k \in \{0, \ldots, d_i - 1\}, \\ R_{j(t+k)} \geq r_i \ \forall k \in \{0, \ldots, d_i - 1\}\} \end{aligned} \tag{17}$$

where the deadline timeslot $D_i = s_i + d_i + w_i$ includes the pod's execution duration and temporal slack for scheduling flexibility.

2) Objective Function: The optimization objective minimizes total carbon emissions across all scheduled pods:

$$\text{minimize} \sum_{i \in \mathcal{P}} \sum_{(j,t) \in \mathcal{V}_i} e_{ijt} \cdot x_{ijt} \tag{18}$$

where e_{ijt} represents the total carbon emissions for executing pod i on node j starting at timeslot t.

3) Carbon Emissions Model: The carbon emissions e_{ijt} incorporate both operational and embodied components over the pod's execution duration d_i:

$$e_{ijt} = \sum_{k=0}^{d_i - 1} (op_{ijt+k} + emb_{ij}) \tag{19}$$

where operational emissions $op_{ijt+k} = p_{ij} \cdot \text{CI}_{j,t+k}$ depend on power consumption p_{ij} and carbon intensity forecasts $\text{CI}_{j,t+k}$, and embodied emissions per timeslot are $emb_{ij} = \frac{\text{EC}_j}{L_j \cdot u_j}$ based on total embodied carbon EC_j, lifetime L_j, and utilization factor u_j.

4) Constraints: The formulation includes resource capacity constraints ensuring CPU and memory limits are respected, temporal constraints enforcing pod deadlines and scheduling windows, and placement constraints requiring each pod to be scheduled exactly once within its feasible region.

E. Proposed Heuristic Carbon-Aware Scheduler

While the MILP formulation provides optimal solutions, it becomes computationally intractable for large-scale deployments with hundreds of pods and nodes. To address this scalability challenge, we develop a heuristic carbon-aware spatiotemporal shifting algorithm that efficiently finds near-optimal solutions in real-time.

1) Algorithm Design Principles: Our heuristic algorithm operates on four key principles:

1) **Priority-driven scheduling**: Pods are processed in deadline order to minimize constraint violations
2) **Spatiotemporal optimization**: The algorithm considers both geographical distribution and temporal flexibility
3) **Comprehensive carbon accounting**: Both operational and embodied emissions influence placement decisions
4) **Real-time adaptability**: The scheduler responds to new pod arrivals and updated carbon intensity forecasts

2) Algorithmic Framework: The core algorithm maintains a priority queue of pending pods ordered by deadline urgency and evaluates all feasible (node, timeslot) combinations to minimize total carbon emissions. For each pod placement candidate, the algorithm computes:

Operational carbon emissions:

$$C^{op}_{ijk} = \widehat{\text{ACI}}_{jk} \times \text{U}_i \times \text{P}_i \tag{20}$$

where $\widehat{\text{ACI}}_{jk}$ is the predicted average carbon intensity for node j in timeslot k, U_i is the execution time of pod i, and P_i is its power consumption.

Embodied carbon emissions:

$$C^{emb}_{ijk} = c^{emb}_j \times \text{U}_i \times \frac{\text{P}_i}{P_{tot}} \tag{21}$$

where $c^{emb}_j = \frac{C^{emb}_j}{L_j}$ is the embodied carbon rate for node j, C^{emb}_j is its total embodied carbon, L_j is its expected lifetime, and P_{tot} represents the total power consumption on the node.

3) Constraint Handling and Feasibility: The algorithm enforces multiple constraint types during placement evaluation:

Resource constraints: CPU and memory requirements must not exceed available node capacity:

$$\overline{\text{CPU}}_i \leq \text{CPU}_j - \sum_{m \in S_{jk}} \overline{\text{CPU}}_m$$

$$\overline{\text{RAM}}_i \leq \text{RAM}_j - \sum_{m \in S_{jk}} \overline{\text{RAM}}_m$$

Utilization constraints: Total pod execution time must fit within available timeslot capacity:

$$\sum_{m \in S_{jk}} \text{U}_m + \text{U}_i \leq \left(1 - \widehat{\text{Util}}_{jk}\right) \times \text{TimeSlotLength}$$

Temporal constraints: Pod completion must respect deadline requirements:

$$\text{End}_k \leq \text{Deadline}_i$$

Algorithm 1 Carbon-Aware Spatiotemporal Shifting Algorithm

Input: Priority queue Q of pods, nodes N, timeslots T
Output: Feasible schedule minimizing carbon emissions

Initialize schedule $S \leftarrow \emptyset$
Initialize resource availability R_{jk} for all $j \in N, k \in T$
Calculate embodied emissions rate $c^{emb}_j \leftarrow \frac{C^{emb}_j}{L_j}$ for all $j \in N$

if new pods arrived **then**
  Add pods to priority queue Q by deadline
end if

if hourly update OR new pods arrive **then**
  for each pod $i \in Q$ **do**
    $C^* \leftarrow \infty, j^* \leftarrow -1, k^* \leftarrow -1$
    for each timeslot $k \in T$ where $\text{End}_k \leq \text{Deadline}_i$ **do**
      for each node $j \in N$ **do**
        if resource constraints satisfied **then**
          if utilization constraints satisfied **then**
            $C^{op}_{ijk} \leftarrow \widehat{\text{ACI}}_{jk} \times \text{U}_i \times \text{P}_i$
            $P_{tot} \leftarrow \sum_{m \in S_{jk}} \text{P}_m + \text{P}_i$
            $C_{tot} \leftarrow C^{op}_{ijk} + c^{emb}_j \times \text{U}_i \times \frac{\text{P}_i}{P_{tot}}$
            if $C_{tot} < C^*$ **then**
              $C^* \leftarrow C_{tot}, j^* \leftarrow j, k^* \leftarrow k$
            end if
          end if
        end if
      end for
    end for
    if $j^* \neq -1$ AND $k^* \neq -1$ **then**
      Schedule pod i on node j^* at timeslot k^*
      Update schedule S and remove pod i from Q
    end if
  end for
end if

4) Computational Complexity: The heuristic algorithm has time complexity $O(|\mathcal{P}| \times |\mathcal{F}| \times |\mathcal{T}|)$ for each scheduling iteration, where $|\mathcal{P}|$ is the number of pending pods, $|\mathcal{F}|$ is the number of nodes, and $|\mathcal{T}|$ is the number of timeslots. This represents a significant improvement over the MILP formulation while maintaining solution quality for practical deployment scenarios.

IV. Experimental Setup

A. Baseline Algorithm 1: MILP-based Oracle

[NA: alternatively move to Proposed Method → Algorithms → MILP Oracle]

The MILP-based oracle provides theoretically optimal solutions by assuming perfect knowledge of all pods (including future arrivals) and formulating the complete scheduling problem as a mixed-integer linear program. This oracle serves as an

upper bound for algorithm performance evaluation and enables analysis of the theoretical optimality gap.

1) Complete Mathematical Model: The complete MILP formulation extends the core problem with detailed constraints:

Placement constraint: Each pod must be scheduled exactly once within its feasible region:

$$\sum_{(j,t)\in\mathcal{V}_i} x_{ijt} = 1 \quad \forall i \in \mathcal{P} \tag{22}$$

CPU capacity constraints: Total CPU usage must not exceed node capacity:

$$\sum_{i\in\mathcal{P}} \sum_{t'\in\mathcal{T}_{it}} c_i \cdot x_{ijt'} \leq C_{jt} \quad \forall j \in \mathcal{F}, \forall t \in \mathcal{T} \tag{23}$$

where $\mathcal{T}_{it} = \{t' \in \mathcal{T} \mid t' \leq t < t' + d_i, (j,t') \in \mathcal{V}_i\}$ represents starting timeslots where pod i would be active at timeslot t.

Memory capacity constraints: Total RAM usage must not exceed node capacity:

$$\sum_{i\in\mathcal{P}} \sum_{t'\in\mathcal{T}_{it}} r_i \cdot x_{ijt'} \leq R_{jt} \quad \forall j \in \mathcal{F}, \forall t \in \mathcal{T} \tag{24}$$

2) Feasibility and Relaxation Techniques: The MILP formulation may become infeasible when resource constraints are over-subscribed or pod deadlines are incompatible with available scheduling windows. We develop a theoretically grounded hierarchy of relaxation techniques that systematically ensure feasibility while preserving solution quality.

Theoretical Foundation: Let $\mathcal{M}$ denote the original MILP formulation. When $\mathcal{M}$ is infeasible, we seek to solve a relaxed problem $\mathcal{M}'$ that: (1) guarantees feasibility, (2) maximizes pod placement, (3) minimizes carbon emissions among solutions with equal placement counts, and (4) maintains computational tractability.

Multi-Objective Optimization with Penalty Functions: Our primary relaxation strategy transforms hard assignment constraints into soft constraints using penalty functions. We introduce slack variables $y_i \in \{0,1\}$ where $y_i = 1$ indicates that pod i remains unplaced. The bi-objective formulation becomes:

$$\begin{aligned}\text{Minimize} \quad Z(\mathbf{x},\mathbf{y}) = \lambda \sum_{i\in\mathcal{P}} y_i \\ + \sum_{i\in\mathcal{P}} \sum_{(j,t)\in\mathcal{V}_i} e_{ijt} \cdot x_{ijt}\end{aligned} \tag{25}$$

where $\lambda \gg \max_{i,j,t} e_{ijt}$ ensures lexicographic optimization priority. The hard assignment constraint is replaced with:

$$\sum_{(j,t)\in\mathcal{V}_i} x_{ijt} + y_i = 1 \quad \forall i \in \mathcal{P} \tag{26}$$

ensuring each pod is either placed exactly once or marked as unplaced. This relaxed formulation always admits a feasible solution while prioritizing maximum pod placement over carbon minimization.

3) Computational Complexity: The MILP-based oracle exhibits fundamentally different computational characteristics compared to the heuristic algorithm. The problem involves $O(|\mathcal{P}| \times |\mathcal{F}| \times |\mathcal{T}|)$ binary decision variables x_{ijt}, though the actual number is constrained by the feasible sets $\mathcal{V}_i$ for each pod i. The resulting binary integer programming problem is NP-hard, with worst-case exponential time complexity $O(2^{|\mathcal{P}|\times|\mathcal{F}|\times|\mathcal{T}|})$.

In practice, modern MILP solvers like CBC employ branch-and-bound algorithms with cutting planes, achieving reasonable performance for moderate problem sizes. However, computational tractability deteriorates rapidly with scale. Our implementation imposes a 30-second time limit and employs the relaxation hierarchy when the full MILP becomes intractable, ensuring system responsiveness while maintaining solution quality where computationally feasible.

The oracle's exponential complexity justifies the heuristic algorithm's polynomial-time approximation for production deployments, while the MILP provides theoretical optimality bounds for performance evaluation and small-scale optimization scenarios.

B. Baseline Algorithm 2: Vanilla K8s Scheduler

C. Testbed Configuration

D. Workload Characteristics

E. Evaluation Metrics

V. Results and Analysis

A. Carbon Reduction: Inclusive and Matched-Set

1) Evaluation Setup and Fairness: To quantify carbon reductions, we evaluate three schedulers—*vanilla* (baseline), *heuristic* (carbon-aware), and *global-optimal* (MILP oracle)—over controlled workload scales (20–200 pods) and time horizons. All algorithms operate under the same constraints (earliest timeslot, CPU/Memory, node capacity/affinity) and the same emissions model (operational + amortized embodied carbon; identical regional carbon-intensity forecasts and node power profiles). We fix seeds where stochasticity is used and report confidence intervals when averaging over multiple runs.

Apples-to-apples comparison is non-trivial because algorithms can schedule different subsets of pods under the same load, confounding emissions with feasibility. To address this, we report **two complementary lenses**:

- **Coverage-Inclusive:** Emissions computed over *all* pods each algorithm successfully schedules. This reflects end-to-end impact (quality *and* coverage) and is the primary view for total environmental effect.
- **Matched-Set (Common Pods Only):** Emissions computed only for pods *scheduled by all three algorithms*. This isolates placement quality by holding the work constant, removing feasibility from the comparison.

Together, these lenses provide a fair and transparent evaluation: inclusive results reflect real impact, while matched-set results isolate the scheduler's carbon-efficiency for identical work.

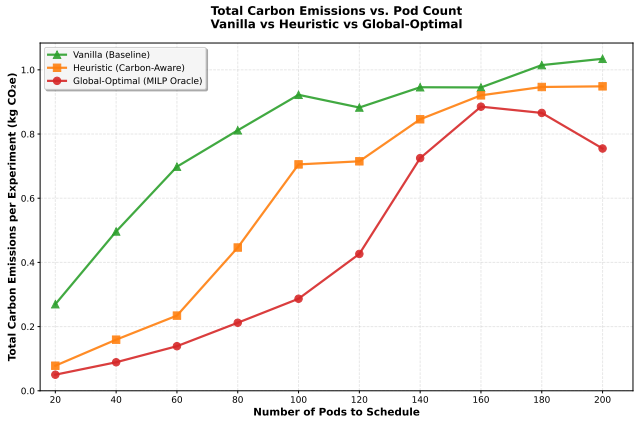

Fig. 4. Total carbon emissions vs. pod count (**coverage-inclusive**). Emissions are computed using a shared model (operational + amortized embodied). Inclusive results reflect each algorithm's *actual* coverage at a given scale.

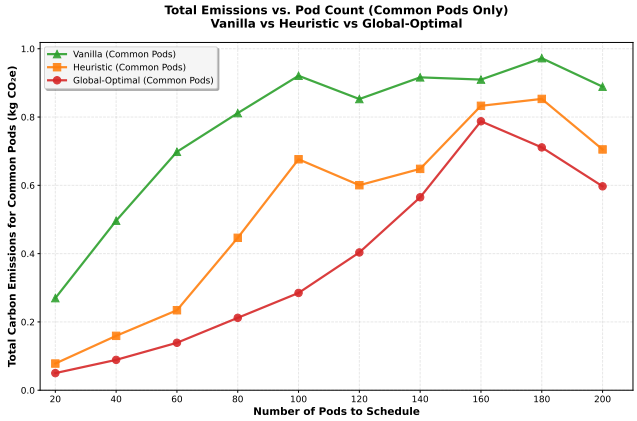

Fig. 5. Total carbon emissions vs. pod count (**matched-set/common pods**). Only pods scheduled by *all three* algorithms are counted, isolating placement quality from feasibility.

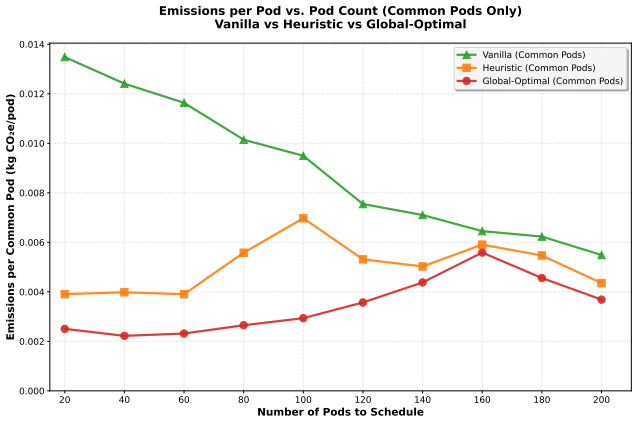

Fig. 6. Emissions per common pod (kg CO_2e/pod) vs. pod count. Lower values indicate higher placement quality for identical work.

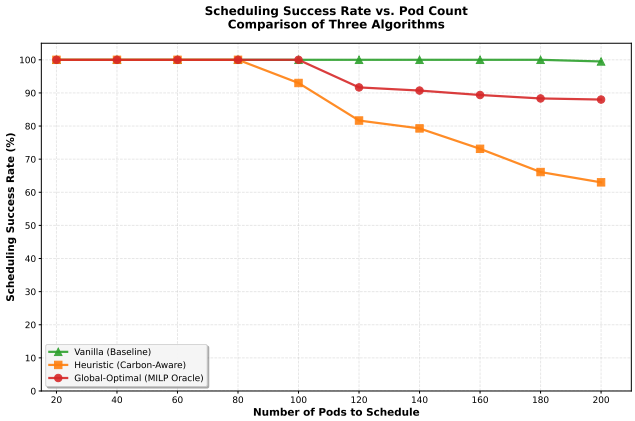

Fig. 7. Scheduling success rate vs. pod count. Higher is better. Used jointly with inclusive emissions to disambiguate feasibility from carbon efficiency.

2) Carbon Reduction Results: Figure 4 shows total emissions versus pod count (coverage-inclusive). As load increases, the carbon-aware schedulers reduce emissions relative to the vanilla baseline; the MILP oracle consistently provides a lower bound, and the heuristic tracks it closely in most scales. Because inclusive totals aggregate whatever each algorithm could schedule, we jointly report success rate in Figure 7 to make coverage explicit.

Figure 5 presents the matched-set (intersection) analysis: totals are restricted to the exact pods scheduled by *all* three algorithms at each scale. This removes coverage differences and reveals the pure placement advantage: global-optimal is best, and the heuristic is typically close, indicating that much of the benefit of the oracle can be captured by a practical heuristic.

Across both views we use the same emissions accounting (operational + embodied). For completeness, per-pod or per-CPU-hour normalizations and operational-only sensitivity analyses can be included in the supplement; the qualitative ordering remains stable under these variants in our experiments.

3) Takeaways and Implications: The dual-lens evaluation yields three practical conclusions: (i) carbon-aware scheduling reduces emissions relative to a vanilla baseline under identical constraints; (ii) a heuristic captures a substantial fraction of the oracle's gains while remaining deployable; and (iii) transparent reporting that pairs coverage-inclusive totals with matched-set analyses provides the fairest and most reproducible comparison across algorithms.

B. Placement Success Rate

Figure 7 shows scheduling success rate versus pod count. At low load, all three methods are near 100%; as load increases, the heuristic remains above the vanilla baseline and close to the MILP oracle. We use this figure alongside the inclusive-emissions and matched-set results to separate feasibility (who gets placed) from carbon efficiency (how those pods are placed).

C. Runtime Cost: Time Complexity

Figure 8 shows the precomputation/runtime cost of the heuristic as a function of pod count, measured using our precompute mode on the same hardware. Runtime grows with workload size but remains practical for the evaluated scales, supporting deployability. By contrast, MILP precomputation

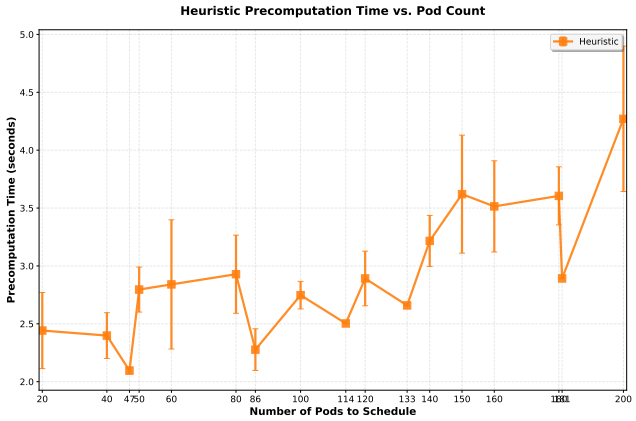

Fig. 8. Precompute/runtime cost vs. pod count for the heuristic scheduler. Error bars (if present) show variability across runs.

for the oracle is substantially heavier and sensitive to solver time limits (not plotted for clarity), which motivates the heuristic for online scheduling while using the oracle primarily as a benchmarking bound.

D. Scalability

[NA:

E. Ablation and Sensitivity

]

VI. Discussion

A. Implications and Limitations

B. Threats to Validity

[NA: internal, external, construct, conclusion, mitigations]

VII. Conclusion and Future Work

References

[1] E. Masanet, A. Shehabi, N. Lei, S. Smith, and J. Koomey, "Recalibrating global data center energy-use estimates," *Science*, vol. 367, no. 6481, pp. 984–986, 2020.

[2] K. Xu, D. Sun, H. Tian, J. Zhang, and K. Chen, "Green: Carbon-efficient resource scheduling for machine learning clusters," in *Proceedings of the 22nd USENIX Symposium on Networked Systems Design and Implementation (NSDI '25)*, 2025. [Online]. Available: https://www.usenix.org/system/files/nsdi25-xu-kaiqiang.pdf

[3] L. Wu, W. A. Hanafy, A. Souza, K. Nguyen, J. Harkes, D. E. Irwin, M. Satyanarayanan, and P. J. Shenoy, "Carbonedge: Leveraging mesoscale spatial carbon-intensity variations for low carbon edge computing," in *Proceedings of the 34th ACM International Symposium on High-Performance Parallel and Distributed Computing (HPDC '25)*, 2025, program: https://hpdc.sci.utah.edu/2025/program.html. [Online]. Available: https://lass.cs.umass.edu/papers/pdf/hpdc25-carbonedge.pdf

[4] P. Wiesner, D. Grinwald, P. Weiß, P. Wilhelm, R. Khalili, and O. Kao, "Quality time: Carbon-aware quality adaptation for energy-intensive services," in *Proceedings of the 16th ACM International Conference on Future Energy Systems (e-Energy '25)*, 2025. [Online]. Available: https://dl.acm.org/doi/10.1145/3679240.3734614

[5] J. Gorka, N. Rhodes, and L. Roald, "Electricityemissions.jl: A framework for the comparison of carbon intensity signals," in *Proceedings of the 16th ACM International Conference on Future Energy Systems (e-Energy '25)*, 2025. [Online]. Available: https://dl.acm.org/doi/10.1145/3679240.3734597

[6] P. Wiesner and O. Kao, "Moving beyond marginal carbon intensity: A poor metric for both carbon accounting and grid flexibility," arXiv preprint, 2025, carbonMetrics Workshop @ ACM SIGMETRICS 2025. [Online]. Available: https://arxiv.org/abs/2507.11377

[7] E. Rodrigues, J. Goldberg, and T. Kosar, "Carbon-aware temporal data transfer scheduling across cloud datacenters," *arXiv preprint*, 2025. [Online]. Available: https://arxiv.org/abs/2506.04117

[8] T. B. Hewage, S. Ilager, M. R. Read, and R. Buyya, "Aging-aware cpu core management for embodied carbon amortization in cloud llm inference," in *Proceedings of the 16th ACM International Conference on Future Energy Systems (e-Energy '25)*, 2025, preprint: https://arxiv.org/abs/2501.15829. [Online]. Available: https://dl.acm.org/doi/10.1145/3679240.3734608

[9] Green Software Foundation, "Software carbon intensity (sci) specification v1.0," Green Software Foundation, Tech. Rep., 2022. [Online]. Available: https://github.com/Green-Software-Foundation/sci

[10] ——, "Embodied emissions (m) guidance," 2023, technical guidance document. [Online]. Available: https://github.com/Green-Software-Foundation/sci/blob/main/Embodied_Emissions_M_Guidance.md

[11] B. Ji, F. Yang, J. C. S. Jones, and X. Zhou, "Advancing environmental sustainability in data centers by proposing carbon depreciation models," *arXiv preprint*, vol. arXiv:2407.12345, 2024. [Online]. Available: https://arxiv.org/abs/2407.12345

[12] L. Zhang, X. Song, and P. Sahoo, "Carbon and reliability-aware computing for heterogeneous data centers," in *Proceedings of the ACM Symposium on Cloud Computing (SoCC)*. ACM, 2025, to appear.

[13] C. Panteleaki, P. Paramanayakam, I. Pentsos *et al.*, "A vertical approach to designing and managing sustainable heterogeneous edge data centers," in *Proceedings of the IEEE International Conference on Edge Computing (EDGE)*. IEEE, 2025, to appear.

[14] Boavizta, "Boavizta methodology for digital services environmental assessment," https://boavizta.org/, 2023, accessed: 2024-01-15.